

LAPORAN PRAKTIKUM
ALGORITMA & STUKTUR DATA
Linked List , Stack & Queue



Di Susun Oleh :

Nama : M. SYARFANI
NIM : 2025573010015
Kelas : TI. 1C
Mata Kuliah : ALGORITMA & STUKTUR DATA
Program Studi : TEKNIK INFORMATIKA
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026/2027

KATA PENGANTAR

Puji syukur kehadiran Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan praktikum dan laporan pada mata kuliah Struktur Data & Algoritma dengan materi ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pembelajaran dan pemahaman mengenai konsep dasar struktur data, khususnya Linked List menggunakan bahasa pemrograman JavaScript dan Node.js. Dalam modul ini, praktikan mempelajari tentang cara kerja Linked List, operasi dasar pada Linked List, serta implementasi Stack menggunakan konsep struktur data tersebut. Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang. Semoga laporan praktikum ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca dalam memahami materi Struktur Data Linked List.

Buket Rata 27 Mei 2026

M. Syarfani

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan	4
1.3 Manfaat.....	5
BAB II PEMBAHASAN.....	6
2.1 Pengertian Linked List.....	6
2.2 Karakteristik Linked List	6
2.3 Insert (Penambahan Data)	6
2.4 Delete (Penghapusan Data)	7
2.5 Traversal.....	7
2.6 Stack.....	7
2.7 Queue.....	7
2.8 Doque.....	8
BAB III PRAKTIKUM	9
3.1 Langkah langkah Praktikum Linked List	9
3.2 Langkah Langkah Praktikum Stack & Queue Lanjutan.....	22
BAB IV PENUTUP	32
4.1 Kesimpulan.....	32
DAFTAR PUSTAKA	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komputer saat ini menuntut kemampuan dalam memahami konsep pemrograman dan pengolahan data secara efisien. Salah satu materi penting dalam ilmu komputer adalah struktur data. Struktur data digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses dengan lebih mudah dan cepat. Salah satu jenis struktur data yang banyak digunakan adalah Linked List. Linked List merupakan struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung. Setiap node menyimpan data dan alamat node berikutnya. Dibandingkan dengan array, Linked List memiliki kelebihan dalam pengelolaan data yang fleksibel karena ukuran data dapat berubah selama program berjalan. Pada praktikum Modul 6 ini, praktikan mempelajari konsep dasar Linked List menggunakan bahasa pemrograman JavaScript dan Node.js. Selain memahami cara kerja Linked List, praktikan juga mempelajari implementasi operasi-operasi dasar seperti penambahan data, penghapusan data, pencarian data, serta penerapan struktur data Stack yang berkaitan dengan Linked List. Melalui praktikum ini diharapkan praktikan dapat memahami konsep struktur data Linked List secara teori maupun praktik sehingga dapat menjadi dasar dalam pengembangan program dan pembelajaran struktur data yang lebih kompleks di masa mendatang.

1.2 Tujuan

- Memahami konsep dasar struktur data Linked List.
- Mengetahui cara kerja node pada Linked List.
- Mempelajari implementasi Linked List menggunakan JavaScript dan Node.js.
- Memahami operasi dasar pada Linked List seperti insert, delete, dan traversal.
- Mempelajari penerapan struktur data Stack menggunakan konsep Linked List.
- Melatih kemampuan praktikan dalam membuat program berbasis struktur data.

1.3 Manfaat

- Menambah pemahaman mengenai struktur data Linked List.
- Melatih kemampuan logika pemrograman dalam pengolahan data dinamis.
- Membantu praktikan memahami implementasi struktur data dalam bahasa pemrograman JavaScript.
- Menjadi dasar pembelajaran untuk materi struktur data yang lebih lanjut.
- Meningkatkan keterampilan praktikan dalam menyusun dan menjalankan program menggunakan Node.js.

BAB II PEMBAHASAN

2.1 Pengertian Linked List

Linked List adalah struktur data linear yang terdiri dari sekumpulan node yang saling terhubung satu sama lain. Setiap node memiliki dua bagian utama, yaitu data yang digunakan untuk menyimpan nilai atau informasi, serta pointer atau link yang berfungsi untuk menunjuk ke node berikutnya. Linked List digunakan untuk menyimpan data secara dinamis sehingga ukuran data dapat bertambah atau berkurang selama program berjalan. Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan data secara berurutan di memori, Linked List menyimpan data secara terpisah namun tetap saling terhubung melalui link pada setiap node. Struktur data Linked List memiliki kelebihan dalam proses penambahan dan penghapusan data karena tidak perlu menggeser elemen seperti pada array. Oleh karena itu, Linked List banyak digunakan dalam berbagai implementasi struktur data lain seperti Stack, Queue, dan Graph.

2.2 Karakteristik Linked List

Karakteristik Linked List adalah ciri-ciri atau sifat yang dimiliki oleh struktur data Linked List. Karakteristik ini membedakan Linked List dengan struktur data lainnya seperti array. Linked List memiliki cara penyimpanan data yang lebih fleksibel karena menggunakan node-node yang saling terhubung. Selain itu, Linked List juga memungkinkan proses penambahan dan penghapusan data dilakukan dengan lebih mudah tanpa harus menggeser seluruh data yang ada.

2.3 Insert (Penambahan Data)

Insert adalah proses menambahkan node atau data baru ke dalam Linked List. Penambahan data dapat dilakukan di awal, di tengah, maupun di akhir Linked List. Pada proses ini, node baru akan dihubungkan dengan node lainnya menggunakan pointer atau link sehingga susunan Linked List tetap terhubung dengan baik. Operasi insert digunakan agar data pada Linked List dapat bertambah secara dinamis sesuai kebutuhan program.

2.4 Delete (Penghapusan Data)

Delete adalah proses menghapus node atau data tertentu dari Linked List. Penghapusan data dapat dilakukan di awal, di tengah, maupun di akhir Linked List. Pada proses ini, hubungan antar node akan diperbarui agar Linked List tetap saling terhubung dengan benar setelah data dihapus. Operasi delete digunakan untuk mengurangi atau menghapus data yang tidak diperlukan lagi di dalam Linked List.

2.5 Traversal

Traversal adalah proses menelusuri atau menampilkan seluruh data yang terdapat pada Linked List dari node pertama hingga node terakhir. Proses traversal dilakukan dengan mengikuti link atau pointer pada setiap node secara berurutan. Operasi ini digunakan untuk melihat, membaca, atau mengecek isi data yang ada di dalam Linked List.

2.6 Stack

Stack pada Linked List adalah penerapan struktur data Stack dengan menggunakan Linked List sebagai media penyimpanan datanya. Stack merupakan struktur data yang menggunakan konsep LIFO (Last In First Out), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar. Pada Stack yang menggunakan Linked List, proses penambahan dan penghapusan data dilakukan pada bagian atas (top) Stack. Operasi dasar pada Stack meliputi push untuk menambahkan data dan pop untuk menghapus data. Penggunaan Linked List pada Stack membuat ukuran data menjadi lebih fleksibel karena dapat bertambah dan berkurang secara dinamis.

2.7 Queue

Queue adalah struktur data linear yang bekerja dengan prinsip FIFO (First In, First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini mirip seperti antrian di kasir atau loket, di mana orang yang datang lebih dulu akan dilayani lebih dulu. Struktur data ini sangat cocok digunakan untuk mengelola data yang memerlukan sistem antrian secara teratur dan berurutan, seperti pada printer, sistem operasi, dan berbagai aplikasi layanan lainnya.

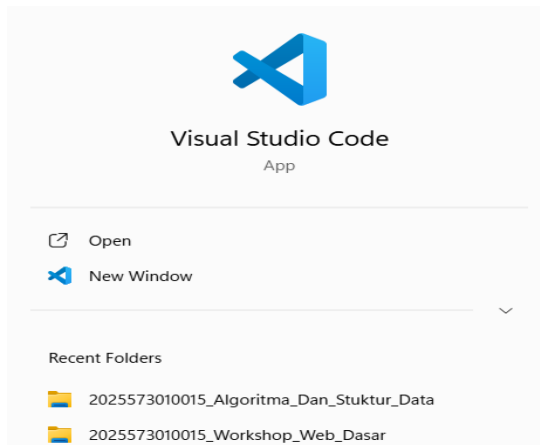
2.8 Deque

Deque (Double Ended Queue) adalah struktur data yang memungkinkan proses penambahan dan penghapusan elemen dilakukan dari kedua ujung antrian, yaitu bagian depan (front) dan belakang (rear). Karena fleksibilitasnya yang lebih tinggi dibandingkan Queue biasa, Deque banyak digunakan dalam berbagai aplikasi yang memerlukan akses data dari dua arah secara efisien.

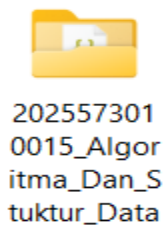
BAB III PRAKTIKUM

3.1 Langkah langkah Praktikum Linked List

1. Langkah Pertama yang dilakukan adalah membuka Visual Studio Code ,



2. Kemudian Pilih Folder yang sesuai dengan nama repository nya , Lalu select folder ,



3. Setelah itu , Buka Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)
$ cd 2025573010015_Algoritma_Dan_Stuktur_Data
```

4. Kemudian buat folder baru praktikum-6, untuk menyimpan semua file di dalam nya ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)
$ mkdir praktikum-6
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)
$ cd praktikum-6
```

5. Lalu , inisialisasi npm project

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ npm init -y
```

6. Di dalam folder praktikum-6 , buat file baru 01-Linked-list.js

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ touch 01-linked-list.js
```

7. Setelah itu , Masuk ke dalam file 01-linked-list.js , Ketik Kode tersebut ,

```
1  class Node {
2    constructor(data) {
3      this.data = data;
4      this.next = null;
5    }
6  }
7
8  class LinkedList {
9    constructor() {
10     this.head = null;
11     this.size = 0;
12   }
13
14   // Tambah data di akhir
15   append(data) {
16     const newNode = new Node(data);
17
18     if (!this.head) {
19       this.head = newNode;
20     } else {
21       let current = this.head;
22
23       while (current.next) {
24         current = current.next;
25       }
26
27       current.next = newNode;
28     }
29
30     this.size++;
31   }
32 }
```

```
34   prepend(data) {
35     const newNode = new Node(data);
36
37     newNode.next = this.head;
38     this.head = newNode;
39
40     this.size++;
41   }
42
43   // Insert data di index tertentu
44   insertAt(data, index) {
45     if (index < 0 || index > this.size) {
46       return false;
47     }
48
49     if (index === 0) {
50       this.prepend(data);
51       return true;
52     }
53
54     const newNode = new Node(data);
55
56     let current = this.head;
57     let previous = null;
58     let i = 0;
59
60     while (i < index) {
61       previous = current;
62       current = current.next;
63       i++;
64     }
65
66     newNode.next = current;
67     previous.next = newNode;
68 }
```

8. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
• $ node 01-linked-list.js
=== Append ===
[10] → [20] → [30] → [40] (size: 4)

=== Prepend ===
[5] → [10] → [20] → [30] → [40] (size: 5)

=== Insert di indeks 2 ===
[5] → [10] → [15] → [20] → [30] → [40] (size: 6)

=== Search ===
Indeks nilai 20: 3
Indeks nilai 99: -1

=== Delete 20 ===
[5] → [10] → [15] → [30] → [40] (size: 5)

=== Reverse ===
[40] → [30] → [15] → [10] → [5] (size: 5)

=== getAt ===
Data pada index 2: 15

=== deleteAt ===
[40] → [15] → [10] → [5] (size: 4)
```

9. Selanjut nya , buat file baru di dalam Praktikum-6 , nama file 02-11-lanjutan.js ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ touch 02-11-lanjutan.js
```

10. Lalu , Masuk ke dalam file 02-11-Lanjutan.js , dan ketik kode tersebut ,

```
1  class Node {
2      constructor(d) {
3          this.data = d;
4          this.next = null;
5      }
6  }
7
8  function adaSiklus(head) {
9      let lambat = head;
10     let cepat = head;
11     while (cepat && cepat.next) {
12         lambat = lambat.next;
13         cepat = cepat.next.next;
14         if (lambat === cepat) return true;
15     }
16     return false;
17 }
18
19 function mergeTerurut(head1, head2) {
20     const dummy = new Node(0);
21     let current = dummy;
22     while (head1 && head2) {
23         if (head1.data <= head2.data) {
24             current.next = head1;
25             head1 = head1.next;
26         } else {
27             current.next = head2;
28             head2 = head2.next;
29         }
30         current = current.next;
31     }
32     current.next = head1 || head2;
33     return dummy.next;
34 }
35
36 function buatList(arr) {
37     if (!arr.length) return null;
38     const head = new Node(arr[0]);
39     let cur = head;
40     for (let i = 1; i < arr.length; i++) {
41         cur.next = new Node(arr[i]);
42         cur = cur.next;
43     }
44     return head;
45 }
46
47 function cetakList(head) {
48     let s = "";
49     let cur = head;
50     while (cur) {
51         s += cur.data + " ";
52         cur = cur.next;
53     }
54     console.log(s);
55 }
56
57 const A = buatList([1, 2, 3, 4, 5]);
58 console.log("Ada siklus (seharusnya false):", adaSiklus(A));
59
60 A.next.next.next.next.next = A.next;
61 console.log("Ada siklus (seharusnya true) :", adaSiklus(A));
62
63 const L1 = buatList([1, 3, 5, 7, 9]);
64 const L2 = buatList([2, 4, 6, 8, 10]);
65 console.log("List 1:");
66 cetakList(L1);
67 console.log("List 2:");
68 cetakList(L2);
69 const merged = mergeTerurut(L1, L2);
70 console.log("Merged:");
71 cetakList(merged);
```

```
35 function buatList(arr) {
36     if (!arr.length) return null;
37     const head = new Node(arr[0]);
38     let cur = head;
39     for (let i = 1; i < arr.length; i++) {
40         cur.next = new Node(arr[i]);
41         cur = cur.next;
42     }
43     return head;
44 }
45
46 function cetakList(head) {
47     let s = "";
48     let cur = head;
49     while (cur) {
50         s += cur.data + " ";
51         cur = cur.next;
52     }
53     console.log(s);
54 }
55
56 const A = buatList([1, 2, 3, 4, 5]);
57 console.log("Ada siklus (seharusnya false):", adaSiklus(A));
58
59 A.next.next.next.next.next = A.next;
60 console.log("Ada siklus (seharusnya true) :", adaSiklus(A));
61
62 const L1 = buatList([1, 3, 5, 7, 9]);
63 const L2 = buatList([2, 4, 6, 8, 10]);
64 console.log("List 1:");
65 cetakList(L1);
66 console.log("List 2:");
67 cetakList(L2);
68 const merged = mergeTerurut(L1, L2);
69 console.log("Merged:");
70 cetakList(merged);
```

11. Selanjut nya , Simpan dan jalankan ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ node 02-11-lanjutan.js
Ada siklus (seharusnya false): false
Ada siklus (seharusnya true) : true

List 1:
  [1]→[3]→[5]→[7]→[9]
List 2:
  [2]→[4]→[6]→[8]→[10]
Merged:
  [1]→[2]→[3]→[4]→[5]→[6]→[7]→[8]→[9]→[10]
```

Latihan 1: Tambah Fitur Linked List

```
267 // ===Latihan 1 menambah fitur Linked list=== //
268 console.log("\n=== getAt ===");
269 console.log("Data pada index 2:", ll.getAt(2));
270
271 console.log("\n=== deleteAt ===");
272 ll.deleteAt(1);
273 ll.print();
274
275 console.log("\n=== indexOf ===");
276 console.log("Index data 30:", ll.indexOf(30));
277 console.log("Index data 100:", ll.indexOf(100));
278
279 console.log("\n=== isEmpty ===");
280 console.log(ll.isEmpty());
281
282 console.log("\n=== toArray ===");
283 console.log(ll.toArray());
284
285 console.log("\n=== clear ===");
286 ll.clear();
287 ll.print();
288
289 console.log("\n=== isEmpty setelah clear ===");
290 console.log(ll.isEmpty());
291
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
● $ node 01-linked-list.js
=== Append ===
[10] → [20] → [30] → [40] (size: 4)

=== Prepend ===
[5] → [10] → [20] → [30] → [40] (size: 5)

=== Insert di indeks 2 ===
[5] → [10] → [15] → [20] → [30] → [40] (size: 6)

=== Search ===
Indeks nilai 20: 3
Indeks nilai 99: -1

=== Delete 20 ===
[5] → [10] → [15] → [30] → [40] (size: 5)

=== Reverse ===
[40] → [30] → [15] → [10] → [5] (size: 5)

=== getAt ===
Data pada index 2: 15

=== deleteAt ===
[40] → [15] → [10] → [5] (size: 4)
```

Latihan 2 : Implementasi Stack dengan Linked List

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ touch 03-stack-11.js.
```

```
1  class Node {
2      constructor(data) {
3          this.data = data;
4          this.next = null;
5      }
6  }
7
8  // Linked List
9  class LinkedList {
10     constructor() {
11         this.head = null;
12         this.length = 0;
13     }
14
15     // menambah di awal (O(1))
16     prepend(data) {
17         const newNode = new Node(data);
18
19         newNode.next = this.head;
20         this.head = newNode;
21
22         this.length++;
23     }
24
25     // kemudian hapus node pertamaH (O(1))
26     removeFirst() {
27         if (!this.head) {
28             return null;
29         }
30
31         const removedData = this.head.data;
32
33         this.head = this.head.next;
34
35         this.length--;
```

```
41     getFirst() {
42         return this.head ? this.head.data : null;
43     }
44
45     isEmpty() {
46         return this.length === 0;
47     }
48
49     size() {
50         return this.length;
51     }
52
53     print() {
54         if (!this.head) {
55             console.log("[Stack kosong]");
56             return;
57         }
58
59         let current = this.head;
60         let result = "";
61
62         while (current) {
63             result += current.next ? `[${current.data}] → ` : `[${current.data}]`;
64             current = current.next;
65         }
66
67         console.log(result);
68     }
69 }
70
71 }
```



```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ node 03-stack-11.js
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ node 03-stack-11.js
=== PUSH ===
[Menulis JavaScript] → [Menulis CSS] → [Menulis HTML] → [Buka VS Code]

=== PEEK ===
Data teratas: Menulis JavaScript

=== SIZE ===
Jumlah data: 4

=== POP ===
Undo: Menulis JavaScript
[Menulis CSS] → [Menulis HTML] → [Buka VS Code]

=== ISEMPY ===
false

=== SIMULASI UNDO ===
Aksi: Ketik Halo
Aksi: Ketik Dunia
Aksi: Hapus Dunia
Aksi: Tambah JavaScript

Isi Stack:
[Tambah JavaScript] → [Hapus Dunia] → [Ketik Dunia] → [Ketik Halo]

Undo 1: Tambah JavaScript
Undo 2: Hapus Dunia

Stack setelah undo:
[Ketik Dunia] → [Ketik Halo]
```

12. Selanjut nya , Buatkan folder tugas dan file tugas

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ mkdir tugas
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

Tugas-1.js

```
1  class Node {
2      constructor(data) {
3          this.data = data;
4
5          this.next = null;
6
7          this.prev = null;
8      }
9  }
10
11 class DoublyLinkedList {
12     constructor() {
13         this.head = null;
14
15         this.tail = null;
16
17         this.size = 0;
18     }
19
20     append(data) {
21         const newNode = new Node(data);
22
23         if (!this.head) {
24             this.head = newNode;
25             this.tail = newNode;
26         } else {
27             this.tail.next = newNode;
28
29             newNode.prev = this.tail;
30             this.tail = newNode;
31         }
32
33         this.size++;
34     }
35 }
```

```
36     prepend(data) {
37         const newNode = new Node(data);
38
39         if (!this.head) {
40             this.head = newNode;
41             this.tail = newNode;
42         } else {
43             newNode.next = this.head;
44
45             this.head.prev = newNode;
46
47             this.head = newNode;
48         }
49
50         this.size++;
51     }
52
53     insertAt(data, index) {
54         if (index < 0 || index > this.size) {
55             return false;
56         }
57
58         if (index === 0) {
59             this.prepend(data);
60             return true;
61         }
62
63         if (index === this.size) {
64             this.append(data);
65             return true;
66         }
67
68         const newNode = new Node(data);
69
70         let current = this.head;
71         let i = 0;
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6/tugas (main)
$ node tugas-1.js

=== PREPEND ===
Forward : [5] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ [40]
Backward: [40] ⇄ [30] ⇄ [20] ⇄ [10] ⇄ [5]

=== INSERT AT INDEX 2 ===
Forward : [5] ⇄ [10] ⇄ [15] ⇄ [20] ⇄ [30] ⇄ [40]
Backward: [40] ⇄ [30] ⇄ [20] ⇄ [15] ⇄ [10] ⇄ [5]

=== DELETE DATA 20 ===
Forward : [5] ⇄ [10] ⇄ [15] ⇄ [30] ⇄ [40]
Backward: [40] ⇄ [30] ⇄ [15] ⇄ [10] ⇄ [5]

=== REVERSE ===
Forward : [40] ⇄ [30] ⇄ [15] ⇄ [10] ⇄ [5]
Backward: [5] ⇄ [10] ⇄ [15] ⇄ [30] ⇄ [40]

=== SIZE ===
Jumlah node: 5
```

Tugas-2.js

```
1  class Node {
2      constructor(data) {
3          this.data = data;
4          this.next = null;
5      }
6  }
7
8  function buatLinkedList(arr) {
9      if (arr.length === 0) return null;
10
11      let head = new Node(arr[0]);
12      let current = head;
13
14      for (let i = 1; i < arr.length; i++) {
15          current.next = new Node(arr[i]);
16          current = current.next;
17      }
18
19      return head;
20  }
21
22  function printLinkedList(head) {
23      let current = head;
24      let result = "";
25
26      while (current) {
27          result += current.data ? `${current.data} → ` : `${current.data}`;
28
29          current = current.next;
30      }
31
32      console.log(result);
33  }
34
```

```
35  function palindromLL(head) {
36      let arr = [];
37      let current = head;
38
39      // Masukkan semua data ke array
40      while (current) {
41          arr.push(current.data);
42          current = current.next;
43      }
44
45      let kiri = 0;
46      let kanan = arr.length - 1;
47
48      while (kiri < kanan) {
49          if (arr[kiri] !== arr[kanan]) {
50              return false;
51          }
52
53          kiri++;
54          kanan--;
55      }
56
57      return true;
58  }
59
60  function hapusNDariAkhir(head, n) {
61      let dummy = new Node(0);
62      dummy.next = head;
63
64      let fast = dummy;
65      let slow = dummy;
66
67      for (let i = 0; i <= n; i++) {
68          fast = fast.next;
69      }
70
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-6/tugas (main)
$ node tugas-2.js
=== PALINDROM LINKED LIST ===
[1] → [2] → [3] → [2] → [1]
Palindrome: true

[1] → [2] → [2] → [1]
Palindrome: true

[1] → [2] → [3] → [4]
Palindrome: false

=== HAPUS N DARI AKHIR ===
[1] → [2] → [3] → [4] → [5]
Setelah hapus:
[1] → [2] → [3] → [5]

[10] → [20] → [30] → [40]
Setelah hapus:
[10] → [20] → [30]

[7] → [8] → [9]
Setelah hapus:
[8] → [9]

=== TENGAH LINKED LIST ===
[1] → [2] → [3] → [4] → [5]
Node tengah: 3

[10] → [20] → [30] → [40] → [50] → [60]
Node tengah: 40

[100] → [200] → [300]
Node tengah: 200
```

3.2 Langkah Langkah Praktikum Stack & Queue Lanjutan

1. Langkah pertama adalah membuat folder praktikum-7 , kemudian masuk ke dalam folder tersebut.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data
$ mkdir praktikum-7
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)
$ cd praktikum-7
```

2. Selanjutnya , Lakukan Inisialisasi Npm Project.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)
$ npm init -y
```

3. Langkah berikutnya , buat file baru 01-stack-II.js ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)
$ touch 01-stack-II.js
```

4. Kemudian , Masuk kedalam file tersebut , ketik kode ini di dalam nya ,

```
1  class Node {
2      constructor(d) {
3          this.data = d;
4          this.next = null;
5      }
6  }
7
8  class Stack {
9      constructor() {
10         this.top = null;
11         this.size = 0;
12     }
13
14     push(data) {
15         const node = new Node(data);
16         node.next = this.top;
17         this.top = node;
18         this.size++;
19     }
20
21     pop() {
22         if (this.isEmpty()) {
23             return null;
24         }
25
26         const val = this.top.data;
27         this.top = this.top.next;
28         this.size--;
29         return val;
30     }
31
32     peek() {
33         return this.top ? this.top.data : null;
34     }
35 }
```

```

36     isEmpty() {
37         return this.size === 0;
38     }
39
40     print() {
41         let s = "TOP → ";
42         let cur = this.top;
43
44         while (cur) {
45             s += `${cur.data} `;
46             cur = cur.next;
47         }
48
49         console.log(s);
50     }
51 }
52
53 function validasiKurung(ekspresi) {
54     const stack = new Stack();
55     const buka = "([{";
56     const tutup = ")]}";
57     const pasangan = {
58         ")": "(",
59         "}": "{",
60         "]" : "[",
61     };
62
63     for (const ch of ekspresi) {
64         if (buka.includes(ch)) {
65             stack.push(ch);
66         } else if (tutup.includes(ch)) {
67             if (stack.isEmpty() || stack.pop() !== pasangan[ch]) {
68                 return false;
69             }
70         }
71     }

```

5. Simpan Kode tersebut , lalu jalan kan di terminal untuk melihat hasil nya ,

```

$ node 01-stack-II.js
=== Validasi Kurung ===
{[(())]} → VALID ✓
(2+3)*[4-1] → VALID ✓
((a+b) → TIDAK VALID ✗
)( → TIDAK VALID ✗
{{{}}} → VALID ✓

=== Evaluasi Postfix ===
3 4 + 2 * = 14
5 1 2 + 4 * + 3 - = 14
2 3 4 * + = 14

```

6. Selanjutnya , buat file baru 02-queue-II.js di dalam folder praktikum-7.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)
$ touch 02-queue-II.js
```

7. Berikutnya , ketik kode tersebut di dalam file 02-queue-II.js yang telah di buat sebelum nya ,

```
1  class Node {
2    constructor(d) {
3      this.data = d;
4      this.next = null;
5    }
6  }
7  class Queue {
8    constructor() {
9      this.head = null;
10     this.tail = null;
11     this.size = 0;
12   }
13   enqueue(data) {
14     const node = new Node(data);
15     if (!this.tail) {
16       this.head = this.tail = node;
17     } else {
18       this.tail.next = node;
19       this.tail = node;
20     }
21     this.size++;
22   }
23   dequeue() {
24     if (this.isEmpty()) return null;
25     const val = this.head.data;
26     this.head = this.head.next;
27     if (!this.head) this.tail = null;
28     this.size--;
29     return val;
30   }
31   front() {
32     return this.head ? this.head.data : null;
33   }
34   isEmpty() {
35     return this.size === 0;
36   }
37   print() {
38     let s = "FRONT → ";
39     cur = this.head;
40     while (cur) {
41       s += `${cur.data} `;
42       cur = cur.next;
43     }
44     console.log(" ", s.trim(), "← BACK");
45   }
46 }
47
48 function bfsGrid(grid, startR, startC) {
49   const rows = grid.length;
50   const cols = grid[0].length;
51   const visited = Array.from({ length: rows }, () => Array(cols).fill(false));
52   const queue = new Queue();
53   const arah = [
54     [0, 1],
55     [0, -1],
56     [1, 0],
57     [-1, 0],
58   ];
59
60   queue.enqueue([startR, startC]);
61   visited[startR][startC] = true;
62   let level = 0;
63   while (!queue.isEmpty()) {
64     const levelSize = queue.size;
65     process.stdout.write(` level ${level}: `);
66     for (let i = 0; i < levelSize; i++) {
67       const [r, c] = queue.dequeue();
68       process.stdout.write(`${r},${c} `);
69       for (const [dr, dc] of arah) {
70         const nr = r + dr;
71         const nc = c + dc;
72         if (

```


8. Kemudian , Simpan file tersebut . lalu jalankan di terminal untuk melihat hasil nya ,

```
$ node 02-queue-11.js
=== Queue Demonstrasi ===
  FRONT → [Pelanggan-A] [Pelanggan-B] [Pelanggan-C] ← BACK
Dilayani: Pelanggan-A
  FRONT → [Pelanggan-B] [Pelanggan-C] [Pelanggan-D] ← BACK

=== BFS pada Grid (. = bisa jalan, # = tembok) ===
Level 0: (0,0)
Level 1: (0,1) (1,0)
Level 2: (0,2) (2,0)
Level 3: (1,2) (3,0)
Level 4: (2,2) (3,1)
Level 5: (2,3)
Level 6: (2,4) (3,3)
Level 7: (3,4) (1,4)
Level 8: (0,4)
```

9. Selanjutnya , Buatkan file baru 03-deque.js di dalam folder praktikum-7.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)
$ touch 03-deque.js
```

10. Lalu , ketik kode ini di dalam file 03-deque.js ,

```
1  class Node {
2    constructor(d) {
3      this.data = d;
4      this.next = null;
5      this.prev = null;
6    }
7  }
8  class Deque {
9    constructor() {
10     this.front = null;
11     this.back = null;
12     this.size = 0;
13   }
14   addFront(data) {
15     const n = new Node(data);
16     if (!this.front) {
17       this.front = this.back = n;
18     } else {
19       n.next = this.front;
20       this.front.prev = n;
21       this.front = n;
22     }
23     this.size++;
24   }
25   addBack(data) {
26     const n = new Node(data);
27     if (!this.back) {
28       this.front = this.back = n;
29     } else {
30       n.prev = this.back;
31       this.back.next = n;
32       this.back = n;
33     }
34   }
35 }
```

```

25     addBack(data) {
26         const n = new Node(data);
27         if (!this.back) {
28             this.front = this.back = n;
29         } else {
30             n.prev = this.back;
31             this.back.next = n;
32             this.back = n;
33         }
34         this.size++;
35     }
36     removeFront() {
37         if (!this.front) return null;
38         const v = this.front.data;
39         this.front = this.front.next;
40         if (this.front) this.front.prev = null;
41         else this.back = null;
42         this.size--;
43         return v;
44     }
45     removeBack() {
46         if (!this.back) return null;
47         const v = this.back.data;
48         this.back = this.back.prev;
49         if (this.back) this.back.next = null;
50         else this.front = null;
51         this.size--;
52         return v;
53     }
54     peekFront() {
55         return this.front ? this.front.data : null;
56     }
57     peekBack() {
58         return this.back ? this.back.data : null;
59     }
60     isEmpty() {

```

11. Simpan kode tersebut , Lalu jalankan di terminal untuk melihat hasil nya ,

```

$ node 03-deque.js
FRONT→ [0]⇄[1]⇄[2]⇄[3] ←BACK
Remove back : 3
Remove front: 0
FRONT→ [1]⇄[2] ←BACK

=== Sliding Window Maximum ===
Array : [
  1, 3, -1, -3,
  5, 3, 6, 7
]
k=3 : [ 3, 3, 5, 5, 6, 7 ]

```

12. Buat folder baru Tugas , di dalam praktikum-7 ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)  
$ mkdir tugas
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-7 (main)  
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

Tugas 1 : Simulasi Sistem Antrian RS

```
1  class Pasien {
2      constructor(id, nama, prioritas, waktuDaftar) {
3          this.id = id;
4          this.nama = nama;
5          this.prioritas = prioritas; // darurat / biasa
6          this.waktuDaftar = waktuDaftar;
7      }
8  }
9
10 class Queue {
11     constructor() {
12         this.items = [];
13     }
14
15     enqueue(data) {
16         this.items.push(data);
17     }
18
19     dequeue() {
20         if (this.isEmpty()) {
21             return null;
22         }
23         return this.items.shift();
24     }
25
26     isEmpty() {
27         return this.items.length === 0;
28     }
29
30     size() {
31         return this.items.length;
32     }
33
34     tampilkan() {
35         return this.items.map((pasien) => `${pasien.nama} (${pasien.prioritas})`);
36     }
37 }
```

```
39 // Class Antrian Rumah Sakit
40 class AntrianRS {
41     constructor() {
42         this.antrianDarurat = new Queue();
43         this.antrianBiasa = new Queue();
44     }
45
46     // pendaftaran pasien
47     daftar(pasien) {
48         if (pasien.prioritas === "darurat") {
49             this.antrianDarurat.enqueue(pasien);
50         } else {
51             this.antrianBiasa.enqueue(pasien);
52         }
53
54         console.log(`${pasien.nama} berhasil didaftarkan (${pasien.prioritas})`);
55     }
56
57     // Proses Layani pasien
58     layani() {
59         let pasien;
60
61         if (!this.antrianDarurat.isEmpty()) {
62             pasien = this.antrianDarurat.dequeue();
63         } else if (!this.antrianBiasa.isEmpty()) {
64             pasien = this.antrianBiasa.dequeue();
65         } else {
66             console.log("Tidak ada pasien dalam antrian.");
67             return;
68         }
69
70         console.log(
71             `Melayani Pasien -> ID: ${pasien.id}, Nama: ${pasien.nama}, Prioritas: ${pasien.prioritas}`,
72         );
73     }
74 }
```

```
$ node tugas-1.js
  'Farah (darurat)',
  'Joko (darurat)'
]

=== ANTRIAN BIASA ===
[
  'Andi (biasa)',
  'Budi (biasa)',
  'Citra (biasa)',
  'Gilang (biasa)',
  'Hani (biasa)',
  'Indra (biasa)'
]

Jumlah Darurat : 4
Jumlah Biasa   : 6

===== PROSES PELAYANAN =====
Melayani Pasien -> ID: 4, Nama: Dina, Prioritas: darurat
Melayani Pasien -> ID: 5, Nama: Eko, Prioritas: darurat
Melayani Pasien -> ID: 6, Nama: Farah, Prioritas: darurat
Melayani Pasien -> ID: 10, Nama: Joko, Prioritas: darurat
Melayani Pasien -> ID: 1, Nama: Andi, Prioritas: biasa
Melayani Pasien -> ID: 2, Nama: Budi, Prioritas: biasa
Melayani Pasien -> ID: 3, Nama: Citra, Prioritas: biasa
Melayani Pasien -> ID: 7, Nama: Gilang, Prioritas: biasa
Melayani Pasien -> ID: 8, Nama: Hani, Prioritas: biasa
Melayani Pasien -> ID: 9, Nama: Indra, Prioritas: biasa

Semua pasien telah dilayani.
```

Tugas 2 : Min Stack

```
1  class Stack {
2      constructor() {
3          this.items = [];
4      }
5
6      push(data) {
7          this.items.push(data);
8      }
9
10     pop() {
11         if (this.isEmpty()) {
12             return null;
13         }
14         return this.items.pop();
15     }
16
17     peek() {
18         if (this.isEmpty()) {
19             return null;
20         }
21         return this.items[this.items.length - 1];
22     }
23
24     isEmpty() {
25         return this.items.length === 0;
26     }
27
28     size() {
29         return this.items.length;
30     }
31 }
32
33 class MinStack {
34     constructor() {
35         this.stack = new Stack();
36         this.minStack = new Stack();
37     }
```

```
39     push(value) {
40         this.stack.push(value);
41
42         if (this.minStack.isEmpty() || value <= this.minStack.peek()) {
43             this.minStack.push(value);
44         }
45
46         console.log(`Push: ${value}`);
47     }
48
49     pop() {
50         if (this.stack.isEmpty()) {
51             return null;
52         }
53
54         const removed = this.stack.pop();
55
56         if (removed === this.minStack.peek()) {
57             this.minStack.pop();
58         }
59
60         console.log(`Pop : ${removed}`);
61         return removed;
62     }
63
64     peek() {
65         return this.stack.peek();
66     }
67
68     getMin() {
69         if (this.minStack.isEmpty()) {
70             return null;
71         }
72
73         return this.minStack.peek();
74     }
```

```
• $ node tugas-2.js
=== UJI MIN STACK ===

Push: 5
Push: 3
Push: 7
Push: 2
Stack: [ 5, 3, 7, 2 ]

Minimum saat ini = 2
Pop : 2
Minimum setelah pop = 3
Pop : 7
Minimum setelah pop = 3
```

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung menggunakan pointer atau link. Setiap node memiliki data dan alamat menuju node berikutnya sehingga data dapat disimpan dan dikelola secara dinamis. Dalam praktikum ini, praktikan mempelajari konsep dasar Linked List, karakteristik Linked List, jenis-jenis Linked List, serta operasi-operasi dasar seperti insert, delete, dan traversal. Selain itu, praktikan juga memahami penerapan Stack menggunakan Linked List dengan konsep LIFO (Last In First Out). Linked List memiliki kelebihan dibandingkan array dalam proses penambahan dan penghapusan data karena tidak perlu menggeser elemen data lainnya. Struktur data ini sangat berguna dalam pembuatan program yang membutuhkan pengelolaan data secara fleksibel dan dinamis. Melalui praktikum ini, praktikan dapat memahami cara kerja Linked List baik secara teori maupun implementasi menggunakan JavaScript dan Node.js. Pemahaman mengenai Linked List diharapkan dapat menjadi dasar yang baik untuk mempelajari struktur data dan algoritma yang lebih kompleks di masa mendatang.

DAFTAR PUSTAKA

- Abdul Kadir. 2018. *Dasar Pemrograman JavaScript*. Yogyakarta: Andi Publisher.
- Rosa A.S dan M. Shalahuddin. 2018. *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*. Bandung: Informatika.
- Narasimha Karumanchi. 2017. *Data Structures and Algorithms Made Easy*. New Delhi: CareerMonk Publications.
- Wahana Komputer. 2020. *Belajar Struktur Data dan Algoritma*. Semarang: Andi Offset.
- Rinaldi Munir. 2019. *Algoritma dan Pemrograman*. Bandung: Informatika.
- Dokumentasi Resmi Node.js.